
E220-900T22S(JP)R2

E220-900T22L(JP)

**Firmware ver.2.0 ライブラリ
利用ガイド**

Rev.1

CLEALINK TECHNOLOGY Co., Ltd.

2024-10-25

目次

1	概要	4
2	接続仕様	5
3	ライブラリの機能	6
4	ライブラリサンプルコードのファイル構成	6
5	サンプルコードの実行	7
6	ライブラリの基本設定	7
7	Firmware ver.1.x 互換モードで動作させる場合	8
7.1	firmware.h 内での条件コンパイルの設定	8
7.2	起動時の初期設定	8
7.2.1	初期化処理の流れ	10
7.3	データの送受信	11
7.3.1	データ送信	11
7.3.2	データ受信	11
7.3.3	受信データ用構造体 (FirmwareVersion 0x10, 0x20, 0x28 共通)	11
7.3.4	受信バッファの他ファイルからのアクセス	11
8	Firmware ver.2.0 Strict Mode (厳格モード) で動作させる場合	13
8.1	firmware.h 内での条件コンパイルの設定	13
8.2	起動時の初期設定	14
8.2.1	初期化処理の流れ	16
8.3	データの送受信	16
8.3.1	データ送信	16
8.3.2	データ受信	17
8.3.3	受信データ (マルチパケット) の取得	17
8.3.4	受信データ用構造体定義 (マルチパケット)	17
8.3.5	Strict Mode での受信データ情報の設定	17
8.3.6	マルチパケット受信時のデータ参照方法	18
8.3.7	UART 受信するデータの正誤判定、マルチパケットを判定を行わない場合	18
8.4	設定レジスタ、状態レジスタ値の読み出し	19
8.4.1	設定・状態および個体番号レジスタの読み出し時の応答を格納する構造体	19
8.4.2	データ参照方法	19
8.4.3	ver.2.0 Strict Mode の動作確認について	21

9	API	22
9.1	Methods for ver.1.x Compatible	22
9.1.1	LoadConfigSetting()	22
9.1.2	InitLoRaModule()	22
9.1.3	RecieveFrame()	23
9.1.4	SendFrame()	23
9.1.5	SwitchToNormalMode()	24
9.1.6	SwitchToWORSendingMode()	24
9.1.7	SwitchToWORReceivingMode()	25
9.1.8	SwitchToConfigurationMode()	25
9.1.9	CheckFirmwareVersion()	25
9.1.10	ReadFirmwareVersion()	26
9.2	Methods for ver.2.0	27
9.2.1	init_REG09_0A_Byte()	27
9.2.2	SetConfigStrictMode()	27
9.2.3	Lowvoltage_operation_Enable()	28
9.2.4	Lowvoltage_operation_Disable()	28
9.2.5	StrictModeStart()	29
9.2.6	StrictModeEnd()	29
9.2.7	ReceivePKTaddADRS()	30
9.2.8	ReceivePKTdelADRS()	30
9.2.9	SendPKTaddCS()	31
9.2.10	SendPKTdelCS()	31
9.2.11	ReceivePKTaddCS()	31
9.2.12	ReceivePKTdelCS()	32
9.2.13	ReceivePKTaddSIZE()	32
9.2.14	ReceivePKTdelSIZE()	33
9.2.15	REG09_ByteWrite()	33
9.2.16	REG09_ByteWrite(uint8_t data)	34
9.2.17	REG0A_ByteWrite()	34
9.2.18	REG09_Set_AUX_HoldTime(uint8_t bt)	35
9.2.19	REG0A_Set_CarrierSenseTime(uint8_t ms)	35
9.2.20	Write_RegisterLIB()	35
9.2.21	Read_RegisterLIB()	36
9.2.22	Read_RegisterLIB_s()	36
9.2.23	read_id_num()	37
9.2.24	write_id_num()	37
9.2.25	Check_BufferedReceiveData_Exist()	38

1 概要

E220-900T22S(JP) R2、E220-900T22L (JP) は、外部 MCU との間で UART の入出力によって、LoRa モジュールのパラメータ設定および LoRa の送受信を行うことができます。本ライブラリは、E220-900T22S(JP) R2、E220-900T22L (JP) を MCU に ESP32 を用いて制御するためのものです。

基本事項詳細は製品データシート Firmware ver.2.0 を参照していただく必要があります。また上記 2つの製品は、従来の E220-900T22S(JP) の Firmware ver.1.0、ver.1.2 への後方互換性を持っており、本ライブラリもその仕様に基づきます。

LoRaモジュール評価ボード E220-900T22S (JP) -EV42



LoRaモジュール評価ボード E220-900T22L (JP) -EV82



3 ライブラリの機能

このライブラリでは、以下の機能を提供します。

- ESP32 内蔵 Flash メモリ内の LoRa コンフィグ設定ファイルから設定値読み込み
- E220-900T22S(JP) へのコンフィグ初期設定
- LoRa 送受信
- ver.2.0 Strict Mode の基本 API

4 ライブラリサンプルコードのファイル構成

```

1  esp32_e220900t22s_jp_lib_v2 フォルダ
2
3  └─ firmware.h
4  └─ esp32_e220900t22s_jp_lib_v2.h
5  └─ esp32_e220900t22s_jp_lib_v2.cpp
6
7  └─ example/
8      └─ lora_multi_task_v1x/
9          └─ lora_multi_task_v1x.ino
10         └─ firmware.h
11         └─ esp32_e220900t22s_jp_lib_v2.h
12         └─ esp32_e220900t22s_jp_lib_v2.cpp
13         └─ data/
14             └─ e220900t22s_jp_lora_config.ini
15         └─ config
16             └─ e220900t22s_jp_lora_config.ini
17
18     └─ lora_multi_task_v20/
19         └─ lora_multi_task_v20.ino
20         └─ firmware.h
21         └─ esp32_e220900t22s_jp_lib_v2.h
22         └─ esp32_e220900t22s_jp_lib_v2.cpp
23         └─ data/
24             └─ e220900t22s_jp_config_v20.ini
25         └─ config
26             └─ e220900t22s_jp_config_v20.ini
27             └─ e220900t22s_jp_config_v28.ini

```

5 サンプルコードの実行

サンプルコードは、ver.1.x 後方互換モード（デフォルト）で実行する場合と、ver.2.0 Strict Mode(厳格モード)で実行する場合の2種類があり、それぞれが、以下のフォルダに配置されています。

- lora_multi_task_v1x ver.1.x 後方互換モード（デフォルト）
- lora_multi_task_v20 ver.2.0 Strict Mode（厳格モード）

以降、それぞれのサンプルコードを実行する場合を説明いたします。また、サンプルコードを実行する場合、必要であれば、ライブラリヘッダーファイル内の「E220-900T22S(JP)へのピンアサイン」を回路に合わせて変更してください。

6 ライブラリの基本設定

本ライブラリでは、firmware ver.1.x 互換モードでの動作か ver.2.0 厳格モードでの動作かを指定するために firmware.h 内での条件コンパイルの設定で変更します。

```
1  #define FIRMWARE_VERSION 0x10  // 0x10 as V.1.x Compatible Mode of  
   Ver.2.0  
2                                     // 0x20 for 22SR2 Strict Mode  
3                                     // 0x28 for 22L   Strict Mode
```

esp32_e220900t22s_jp_lib_v2.cpp はクラスライブラリになっておりますので、main.ino など
esp32_e220900t22s_jp_lib_v2.h を include したうえでこの1行を追加しておく必要があります。

上記ファイル構成の中の lora_multi_task_v1x.ino および lora_multi_task_v20.ino 内をご参照ください。

```
1  CLoRa lora;
```

7 Firmware ver.1.x 互換モードで動作させる場合

Firmware ver.1.x 互換モードで動作させる場合のライブラリを使用する方法をサンプルプログラム lora_multi_task_v1x.ino を用いて説明します。

7.1 firmware.h 内での条件コンパイルの設定

LoRa モジュール E220-900T22S(JP)SR2、E220-900T22L(JP) を firmware ver.1.x 互換モードで動作させる設定。

```
1 #define FIRMWARE_VERSION 0x10
```

7.2 起動時の初期設定

起動時の初期設定は、プログラムコードで記述するデフォルト設定で行うか、lora_multi_task_v1x 内の data フォルダに置かれた設定ファイル esp32_e220900t22s_jp_lora_config.ini を「ESP32 Sketch Data Upload」によって ESP32 の Flash メモリに書き込んで置き、起動時に読み出してその設定値を用いることで行います。(設定ファイルを消去するには、data フォルダの .ini ファイルを消去し、「ESP32 Sketch Data Upload」を行ってください。)

esp32_e220900t22s_jp_lora_config.ini 設定例

1 own_address=0	・・・ 自機モジュールアドレスを0
2 baud_rate=9600	・・・ UARTのbaud rateを9600bps
3 bw=125	・・・ BW(帯域幅)を125kHz
4 sf=9	・・・ SF(拡散率)を9
5 subpacket_size=200	・・・ パケット長(ペイロード長)を200byte
6 transmitting_power=13	・・・ 送信出力電力を13dBm
7 own_channel=0	・・・ 自機周波数チャンネルを0
8 rssi_byte_flag=1	・・・ RSSIバイトを有効化
9 transmission_method_type=1	・・・ 固定送信モード
10 wor_cycle=2000	・・・ WORサイクルを2000ms
11 encryption_key=0	・・・ 暗号化キーを0
12 target_address=0	・・・ ターゲットアドレスを0
13 target_channel=0	・・・ ターゲットチャンネルを0

指定していない項目は、下記のプログラム内で設定したデフォルト値が設定されます。

esp32_e220900t22s_jp_lib_v2.cpp 内でのコンフィグデフォルト値設定例

```
1 // priority definition A (1 - 13)
2
3 void CLoRa::SetDefaultConfigValue_into_private_val(){
4     own_address_val = 0x0000;           // 1
5     baud_rate_val = 9600;               // 2
6     bw_val = 125;                       // 3
7     sf_val = 9;                         // +3
8     subpacket_size_val = 200;           // 4
9     rssi_ambient_noise_flag_val = 1;    // 5
10    transmitting_power_val = 13;        // 6
11    own_channel_val = 0x0;               // 7
12    rssi_byte_flag_val = 1;              // 8
13    transmission_method_type_val = 1;   // 9
14    wor_cycle_val = 2000;                //10
15    encryption_key_val = 0;              //11
16
17    target_address_val = 0x0000;         //12
18    target_channel_val = 0x0;            //13
19 }
20
21 // E220-900T22S/L Default definition B (1 - 13)
22
23 const LoRaConfigItem_t default_config = {
24     0x0000, // 1 own_address 0
25     0b011, // 2 baud_rate 9600 bps
26     0b10000, // 3 air_data_rate SF:9 BW:125
27     0b00, // 4 subpacket_size 200
28     0b0, // 5 rssi_ambient_noise_flag 無効
29     0b01, // 6 transmitting_power 13 dBm
30     0x0, // 7 own_channel 0
31     0b0, // 8 rssi_byte_flag 無効
32     0b1, // 9 transmission_method_type 固定送信モード
33     0b011, //10 wor_cycle 2000 ms
34     0x0000, //11 encryption_key 0
35
36     0x0000, //12 target_address 0
37     0x00, //13 target_channel 0
38 }
```

7.2.1 初期化処理の流れ

```
1 // コンフィグモード (M0=1,M1=1) へ移行する
2 lora.SwitchToConfigurationMode();
3
4 // .ini ファイルを使用しない場合
5 // lora.SetDefaultConfigValue(lora.config); // ライブラリ内のコン
   // フィグデフォルト値設定
6
7 // .ini から LoRa設定値を読み込む
8 if (lora.LoadConfigSetting(CONFIG_FILENAME, lora.config)) {
9                                     // Flash メモリ内設定
   ファイルの読み込み
10   SerialMon.printf("\nLoading Configfile failed. The default value is
   set.\n");
11 } else {
12   SerialMon.printf("\nLoading Configfile succeeded.\n");
13 }
14
15 // E220-900T22S(JP) への LoRa初期設定
16 if (lora.InitLoRaModule(lora.config)) {
17   SerialMon.printf("init error\n");
18   return;
19 }
```

最終設定値は、lora.config に格納されます。以下に、コンフィグレーションをアプリケーションの他ファイルからのアクセスするコードを記します。

```
1 lora.config
2 lora.config.own_address
3 lora.air_data_rate
4 : etc
```

7.3 データの送受信

本サンプルプログラムでのデータの送受信は、送信、受信それぞれをマルチタスクで以下の手順で実行します。

送信：LoRaSendTask() Arduino IDE のシリアルモニタからのキー入力を LoRa へ UASRT 送信。

受信：LoRaReceiveTask() LoRa からの UART データを受信し、受信バッファへ格納。

7.3.1 データ送信

マルチタスクで実行される LoRaSendTask() において、Arduino IDE シリアルモニタ上からのキー入力を msg[] に読み込み、以下のように送信関数を呼び出します。関数の詳細は API の Method 部を参照してください。

```
1 lora.SendFrame(lora.config, (uint8_t *)msg, strlen(msg));
```

7.3.2 データ受信

マルチタスクで実行される LoRaReceiveTask() において、LoRa からの UART 受信データを lora.data.recv_data[] に読み込みます。

```
1 lora.ReceiveFrame(&lora.data);
```

7.3.3 受信データ用構造体 (FirmwareVersion 0x10, 0x20, 0x28 共通)

```
1 struct RecvFrameE220900T22SJP_t {  
2     uint8_t recv_data[408];  
3     uint8_t recv_data_len;  
4     int rssi;  
5 };  
6  
7 struct RecvFrameE220900T22SJP_t data;    /** Received Air Data  
    Area */
```

7.3.4 受信バッファの他ファイルからのアクセス

```
1 lora.data.recv_data    // 受信データ格納域  
2 lora.data.recv_data_len // 受信データ長  
3 lora.data.rssi         // 受信データのRSSI値 (rssi_byte_flag  
    が 1 の時に有効)
```

lora_multi_task_v1x 実行例

- 起動～1 パケット受信

```
1 Read config file /e220900t22s_jp_lora_config.ini.
2
3 own_address = 0
4 baud_rate = 9600
5 bw = 125
6 sf = 9
7 subpacket_size = 200
8 transmitting_power = 13
9 own_channel = 0
10 wor_cycle = 2000
11 encryption = 0
12 target_address = 0
13 target_channel = 0
14 rssi_byte_flag = 1
15
16 Loading Configfile succeeded.
17 switch to configuration mode
18 # Command Request
19 0xc0 0x00 0x08 0x00 0x00 0x70 0x21 0x00 0xc3 0x00 0x00
20 # Command Response
21 0xc1 0x00 0x08 0x00 0x00 0x70 0x21 0x00 0xc3 0x00 0x00
22
23 ff 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18
   19 1a 1b 1c 1d be
24 Received Length = 29
25 recv data:
26 hex dump:
27 ff 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18
   19 1a 1b 1c 1d
28 RSSI: -66 dBm
```

8 Firmware ver.2.0 Strict Mode (厳格モード) で動作させる場合

Firmware ver.2.0 Strict Mode で動作させる場合のライブラリの使用方法をサンプルプログラム lora_multi_task_v20.ino を用いて説明します。

8.1 firmware.h 内での条件コンパイルの設定

FirmwareVersion 製品毎に設定を変更します。

- E220-900T22S(JP) R2 は、0x20。
- E220-900T22L(JP) は、0x28。

```
1 #define FIRMWARE_VERSION 0x10 // 0x10 as V.1.x Compatible Mode of  
   Ver.2.0  
2                               // 0x20 for 22SR2 Strict Mode  
3                               // 0x28 for 22L Strict Mode
```

LoRa モジュール E220-900T22S(JP)SR2 を ver.2.0 Strict Mode (厳格モード) での動作させる場合。

```
1 #define FIRMWARE_VERSION 0x20
```

LoRa モジュール E220-900T22L(JP) を ver.2.0 Strict Mode (厳格モード) での動作させる場合。

```
1 #define FIRMWARE_VERSION 0x28
```

この条件コンパイルだけでは、ver.2.x Strict Mode にはなりません。以下の初期化手順を実行するか、

```
1 lora.StrictModeStart();
```

を実行してください。その後、Strict Mode での各種設定を行うことが可能です。

8.2 起動時の初期設定

起動時の初期設定は、プログラムコードで記述するデフォルト設定で行うか、E220-900T22S(JP)R2 の場合は lora_multi_task_v20 内の data フォルダに置かれた設定ファイル esp32_e220900t22s_jp_config_v20.ini を「ESP32 Sketch Data Upload」によって ESP32 の Flash メモリに書き込んで置き、起動時に読み出してその設定値を用いることで行います。E220-900T22L(JP) を接続した場合は、backup フォルダに置かれた esp32_e220900t22s_jp_config_v28.ini を data フォルダにコピーして用いてください。

(設定ファイルを消去するには、data フォルダの .ini ファイルを消去し、「ESP32 Sketch Data Upload」を行ってください。)

esp32_e220900t22s_jp_config_v20.ini 設定例

1	own_address=0	・・・	自機モジュールアドレスを0
2	baud_rate=9600	・・・	UARTのbaud rateを9600bps
3	bw=125	・・・	BW(帯域幅)を125kHz
4	sf=9	・・・	SF(拡散率)を9
5	subpacket_size=200	・・・	パケット長(ペイロード長)を200byte
6	transmitting_power=13	・・・	送信出力電力を13dBm
7	own_channel=0	・・・	自機周波数チャンネルを0
8	rss_i_byte_flag=1	・・・	RSSIバイトを有効化
9	transmission_method_type=1	・・・	固定送信モード
10	wor_cycle=2000	・・・	WORサイクルを2000ms
11	encryption_key=0	・・・	暗号化キーを0
12	target_address=0	・・・	ターゲットアドレスを0
13	target_channel=0	・・・	ターゲットチャンネルを0
14	strict_mode=enable	・・・	strict mode 動作切り替え enable /
	diab le		
15	lowvoltage_operation=enable	・・・	低電圧動作の有効(enable)/無効(
	disab le)		
16	recv_target_address=enable	・・・	受信パケットの宛先アドレス出力の有
	効(enable)/無効(disab le)		
17	send_data_checksum=enable	・・・	MCUからLoRaへのUART送信データに
	CHECKSUMの要/不要		
18	recv_data_checksum=enable	・・・	LoRaからMCUへのUART送信データに
	CHECKSUMの要/不要		
19	recv_data_packetsize=enable	・・・	LoRaからMCUへのUART送信データのパ
	ケット長の要/不要		
20	aux_low_holdtime=512	・・・	受信動作のAUX Low レベル待機時間
21	carriresense_timeout=127	・・・	送信前のキャリアセンス最大待機時間
	の設定		

指定していない項目は、下記のプログラム内で設定したデフォルト値が設定されます。

esp32_e220900t22s_jp_lib_v2.cpp 内でのコンフィグデフォルト値設定例

```

1 // priority definition A (1 - 13)
2
3 void CLoRa::SetDefaultConfigValue_into_private_val(){
4     own_address_val = 0x0000;           // 1
5     baud_rate_val = 9600;               // 2
6     bw_val = 125;                       // 3
7     sf_val = 9;                         // +3
8     subpacket_size_val = 200;           // 4
9     rssi_ambient_noise_flag_val = 1;    // 5
10    transmitting_power_val = 13;         // 6
11    own_channel_val = 0x0;               // 7
12    rssi_byte_flag_val = 1;             // 8
13    transmission_method_type_val = 1;   // 9
14    wor_cycle_val = 2000;                //10
15    encryption_key_val = 0;             //11
16    target_address_val = 0x0000;        //12
17    target_channel_val = 0x0;           //13
18 }
19
20 // E220-900T22S/L Default definition B (1 - 13)
21
22 const LoRaConfigItem_t default_config = {
23     0x0000, // 1 own_address 0
24     0b011, // 2 baud_rate 9600 bps
25     0b10000, // 3 air_data_rate SF:9 BW:125
26     0b00, // 4 subpacket_size 200
27     0b0, // 5 rssi_ambient_noise_flag 無効
28     0b01, // 6 transmitting_power 13 dBm
29     0x0, // 7 own_channel 0
30     0b0, // 8 rssi_byte_flag 無効
31     0b1, // 9 transmission_method_type 固定送信モード
32     0b011, //10 wor_cycle 2000 ms
33     0x0000, //11 encryption_key 0
34     0x0000, //12 target_address 0
35     0x00, //13 target_channel 0
36 #if ( (FIRMWARE_VERSION == 0x20) || (FIRMWARE_VERSION == 0x28) )
37     0b1, // strict_mode
38     0b0, // Lowvoltage_operation enable = 0
39     0b0, // recv_target_address disable
40     0b0, // send_data_checksum disable
41     0b0, // recv_data_checksum disable
42     0b1, // recv_data_packet_size enable
43     0b00, // aux_low_holdtime 00 default:2-3msec
44     0x00, // carriresense_timeout 00 (default)
45 #endif
46 #if FIRMWARE_VERSION == 0x28
47     0b0 // transmitting_power_table 0
48 #endif
49 };

```

8.2.1 初期化処理の流れ

```
1 // コンフィグモード(M0=1,M1=1)へ移行する
2 lora.SwitchToConfigurationMode();
3
4 // .ini ファイルを使用しない場合
5 // lora.SetDefaultConfigValue(lora.config); // ライブラリ内のコ
   // ンフィグデフォルト値設定
6
7 // .ini から LoRa設定値を読み込む
8 if (lora.LoadConfigSetting(CONFIG_FILENAME, lora.config)) {
9     SerialMon.printf("\nLoading Configfile failed. The default value is
   // set.\n");
10 } else {
11     SerialMon.printf("\nLoading Configfile succeeded.\n");
12 }
13
14 lora.init_REG09_0A_Byte();
15
16 // E220-900T22S(JP)へのLoRa初期設定
17 if (lora.InitLoRaModule(lora.config)) {
18     SerialMon.printf("init error\n");
19     return;
20 }
21 if (lora.SetConfigStrictMode()) {
22     SerialMon.printf("Strict mode init error\n");
23     return;
24 }
```

8.3 データの送受信

本サンプルプログラムでのデータの送受信は、送信、受信それぞれをマルチタスクで以下の手順で実行します。

送信：LoRaSendTask() Arduino IDE のシリアルモニタからのキー入力を LoRa へ UASRT 送信。

受信：LoRaReceiveTask() LoRa からの UART データを受信し、受信バッファへ格納。

8.3.1 データ送信

マルチタスクで実行される LoRaSendTask() において、Arduino IDE シリアルモニタ上からのキー入力を msg[] に読み込み、以下のように送信関数を呼び出します。関数の詳細は API の Method 部を参照してください。

```
1 lora.SendFrame(lora.config, (uint8_t *)msg, strlen(msg));
```


8.3.2 データ受信

マルチタスクで実行される `LoRaReceiveTask()` において、LoRa からの UART 受信データを `lora.data.recv_data[]` に読み込み、設定レジスタの設定が必要であれば、マルチパケット情報を提供します。

```
1 lora.ReceiveFrame(&lora.data);
```

8.3.3 受信データ（マルチパケット）の取得

複数のパケット（マルチパケット）を受信したかどうかの判定は、Strict Mode の設定レジスタ bit2 の受信データ列へのパケットサイズ付与指定フラグが Enable のときに有効となります。

Disable の場合は、ESP32 が受信したデータは、LoRaAir 伝送データ 1 パケットの扱いになります。（下記の `RcvPacket[0].recv_multi_pkt_f` は 0x00 になります。）

8.3.4 受信データ用構造体定義（マルチパケット）

```
1 struct LoRaRecvPacket_t {  
2     uint8_t packet_size;  
3     uint16_t packet_address;  
4     uint8_t *packet_data;  
5     int packet_rssi;  
6     int packet_error;  
7     uint8_t recv_multi_pkt_f; // 次に受信したパケット  
    の有無  
8 };  
9  
10 struct LoRaRecvPacket_t RcvPacket[x]; // x : MAX受信パケット数  
    ユーザアプリで変更
```

8.3.5 Strict Mode での受信データ情報の設定

Strict Mode では、LoRa からの受信データに以下の 3 つの設定が用意されています。

- データのサイズ
- チェックサム
- 受信したデータが自機宛か、ブロードキャストされたかを判定する宛先アドレス

マルチパケットを受信する可能性がある場合には、データのサイズを取得する設定が必要です。以下の定義で設定するか、

- `esp32_e220900t22s_jp_config_v20.ini` での定義

・ esp32_e220900t22s_jp_config_v20.cpp 内でのコンフィグデフォルト値の設定

この関数の呼び出しを行ってください。

```
1 lora.ReceivePKTaddSIZE();
```

LoRa から UART 受信するデータの正誤判定、マルチパケットを判定するかしないかは firmware.h 内の条件コンパイルに従います。

```
1 #define RECEIVE_DATA_CHECK 1 // Check chkSum & Received Length In
   Strict Mode
```

8.3.6 マルチパケット受信時のデータ参照方法

1	lora.RcvPacket[x].packet_size	packet size
2	lora.RcvPacket[x].packet_address	受信した 宛先アドレス
3	lora.RcvPacket[x].*packet_data	受信データの lora.data.
	recv_data 内のポインタ	
4	lora.RcvPacket[x].packet_rssi	受信パケットのRSSI値
5	lora.RcvPacket[x].packet_error	サムチェックエラー有無
6	lora.RcvPacket[x].recv_multi_pkt_f	次のパケットの有無

例) 29 バイトを ffff(ブロードキャスト) で受信したデータと、32 バイトを宛先 0000 で受信した例

```
1 lora.RcvPacket[0].packet_size = 29
2 lora.RcvPacket[0].packet_address = ffff
3 lora.RcvPacket[0].packet_data = ff 02 03 04 05 06 07 ...
4 lora.RcvPacket[0].recv_multi_pkt_f = 1
5 lora.RcvPacket[0].packet_rssi = -59
6
7 lora.RcvPacket[1].packet_size = 32
8 lora.RcvPacket[1].packet_address = 0000
9 lora.RcvPacket[1].packet_data = 02 00 00 00 00 00 00 ...
10 lora.RcvPacket[1].recv_multi_pkt_f = 0
11 lora.RcvPacket[1].packet_rssi = -24
```

※ recv_multi_pkt_f は“1”で次のパケットが存在していることを示し、“0”が最終パケットとなります。

8.3.7 UART 受信するデータの正誤判定、マルチパケットを判定を行わない場合

```
1 #define RECEIVE_DATA_CHECK 0 // Check ChkSum & Received Length In
   Strict Mode
```

LoRa 受信関数 `ReceiveFrame()` で、受信データ長とサムチェックの判定をせず、マルチパケット対応処理も省略し、受信データのアクセスは ver.1.x のアクセス方法でも可能です。

```
1    lora.data
2    lora.data.recv_data           // 受信データ格納域
3    lora.data.recv_data_len      // 受信データ長
4    lora.data.rssi               // 受信データのRSSI値
                                   (rssi_byte_flagが1の時に有効)
```

8.4 設定レジスタ、状態レジスタ値の読み出し

8.4.1 設定・状態および個体番号レジスタの読み出し時の応答を格納する構造体

```
1    struct REGISTERVal_t {
2        uint8_t response;
3        uint8_t address;
4        uint8_t bytes;
5        uint8_t val[10];
6        uint8_t error;
7    };
8
9    union REGUnion {
10        REGISTERVal_t REGUniStr;           アクセス用
11        uint8_t data[10];                 ライブラリでの格納用
12    } REGData;
13
14    union REGUnion REGData;
```

8.4.2 データ参照方法

レジスタ読み出しコマンドの応答値の参照は以下の方法になります。

```
1    lora.REGData.REGUniStr.response;
2    lora.REGData.REGUniStr.address;
3    lora.REGData.REGUniStr.bytes;
4    lora.REGData.REGUniStr.val[0];
```

レジスタ実データは、

```
1    lora.REGData.REGUniStr.val[0～];
2    または
3    lora.REGData.data[3～];
```

lora_multi_task_v1x 実行例

- 起動～2 パケット受信

```
1 Read config file /e220900t22s_jp_config_v20.ini.
2
3 own_address = 0
4 baud_rate = 9600
5 bw = 125
6 sf = 9
7 subpacket_size = 200
8 transmitting_power = 13
9 own_channel = 0
10 wor_cycle = 2000
11 encryption = 0
12 target_address = 0
13 target_channel = 0
14 rssi_byte_flag = 1
15 transmission_method_type = 1
16 strict_mode = enable
17 lowvoltage_operation = enable
18 recv_target_address = enable
19 send_data_checksum = enable
20 recv_data_checksum = enable
21 recv_data_packet_size = enable
22 aux_low_holdtime = 512
23 carriresense_timeout = 10
24
25
26 Loading Configfile succeeded.
27
28 c0 0a 01 00 cb
29 ff ff ff
30
31 c0 09 01 00 c8
32 ff ff ff
33
34 c0 09 01 00
35 c1 09 01 00
36
37 switch to configuration mode
38 # Command Request
39 0xc0 0x00 0x08 0x00 0x00 0x70 0x21 0x00 0xc3 0x00 0x00
40 # Command Response
41 0xc1 0x00 0x08 0x00 0x00 0x70 0x21 0x00 0xc3 0x00 0x00
42
43 conf_REG09_Byte = 7d
44
45 c0 09 01 7d
46 c1 09 01 7d
47
```

```

48 c0 0a 01 00 cb
49 05 c1 0a 01 00 cf
50
51 Write ID num Register = c0 e0 01 55 74
52 05 c1 e0 01 55 70
53 Read ID num Register = c1 e0 01 20
54 0x05 0xc1 0xe0 0x01 0x55 0x70
55
56 IDnum Read after write : 55
57
58 conf_REG09_Byte = 7d
59
60 MultiPacket!!! 69
61 len = 36
62 RECEIVE CHECKSUM (0)= 00 8f 8f
63
64 recv data:
65 24 00 00 02 00 00 00 00 00 00 00 00 00 56 00 00 00 00 56 00 00 00 00 00 00
   00 00 00 00 40 46 00 00 00 00
66 RSSI: -65 dBm
67
68
69 RcvPacket[0].packet_size = 32
70 RcvPacket[0].packet_address = 0000
71 RcvPacket[0].packet_data = 02 00 00 00 00 00 00 00 00 00 56 00 00 00 00
   56 00 00 00 00 00 00 00 00 00 00 00 40 46 00 00 00 00 00
72 RcvPacket[0].recv_multi_pkt_f = 1
73 RcvPacket[0].packet_rssi = -81
74
75 RcvPacket[1].packet_size = 29
76 RcvPacket[1].packet_address = ffff
77 RcvPacket[1].packet_data = ff 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e
   0f 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d
78 RcvPacket[1].recv_multi_pkt_f = 1
79 RcvPacket[1].packet_rssi = -65

```

8.4.3 ver.2.0 Strict Mode の動作確認について

main プログラムの lora_multi_task_v20.ino の最上部にある

```
1 #define CHECK_STRICT_EXECUTION 1
```

が“1”の場合、Strict Mode での R E G 0 9 の設定レジスタのいくつかの設定を ON にした状態の動作が確認できます。

- シリアルモニタからの入力待ち

```
1 Input 0, 1, 2, 3, 4 :
```

9 API

9.1 Methods for ver.1.x Compatible

9.1.1 LoadConfigSetting()

ESP32 の内蔵 Flash から、引数 `filename` のファイルを読み込み、引数 `config` の構造体メンバに設定値をセットします。

ESP32 内に設定ファイルが存在しない場合、デフォルト値がセットされます。

Syntax

- type `int`

```
1  lora.LoadConfigSetting(const char *filename, struct  
    LoRaConfigItem_t &config)
```

Parameters

- `filename`: 設定ファイル名
- `config`: 読み込んだ設定値の格納先

Returns

- 0: 成功; 1: 失敗

9.1.2 InitLoRaModule()

E220-900T22S(JP) をコンフィグモード (M0=1,M1=1) へ移行させ、引数 `config` の構造体メンバをもとに LoRa 初期設定を行います。

Syntax

- type `int`

```
1  lora.InitLoRaModule(struct LoRaConfigItem_t &config)
```

Parameters

- `config`: 設定値の格納先

Returns

- 0:成功; 1: 失敗

9.1.3 RecieveFrame()

LoRa 受信を行い、引数 `recv_frame`の構造体メンバへ値を格納します。

Syntax

- type *int*

```
1 lora.RecieveFrame(struct RecvFrameE220900T22SJP_t *recv_frame);
```

Parameters

- `recv_frame`: LoRa 受信データの格納先

Returns

- 0:成功; 1: 失敗

9.1.4 SendFrame()

引数 `send_data`の LoRa 送信を行います。

Syntax

- type *int*

```
1 lora.SendFrame(struct LoRaConfigItem_t &config, uint8_t *send_data,  
    int size);
```

Parameters

- `config`: 設定値の格納先
- `send_data`: 送信データ
- `size`: 送信データサイズ

Returns

- 0:成功; 1: 失敗

9.1.5 SwitchToNormalMode()

ノーマルモード (M0=0,M1=0) へ移行します。

Syntax

- type void

```
1 lora.SwitchToNormalMode(void);
```

Parameters

- none

Returns

- none

9.1.6 SwitchToWORSendingMode()

WOR 送信モード (M0=1,M1=0) へ移行します。

Syntax

- type void

```
1 lora.SwitchToWORSendingMode(void);
```

Parameters

- none

Returns

- none

9.1.7 SwitchToWORReceivingMode()

WOR 受信モード (M0=0,M1=1) へ移行します。

Syntax

- type void

```
1  lora.SwitchToWORReceivingMode(void);
```

Parameters

- none

Returns

- none

9.1.8 SwitchToConfigurationMode()

コンフィグモード (M0=1,M1=1) へ移行します。

Syntax

- type void

```
1  lora.SwitchToConfigurationMode(void);
```

Parameters

- none

Returns

- none

9.1.9 CheckFirmwareVersion()

ライブラリの条件コンパイルの設定 (firmware.h の FIRMWARE_VERSION) と LoRa モジュールの Version が一致しているかどうかのチェックを行います。

Syntax

- type *int*

```
1  lora.CheckFirmwareVersion();
```

Parameters

- none

Returns

- 0:一致; 1: 不一致

9.1.10 ReadFirmwareVersion()

コンフィグレーションモード (mode3 : M0=1,M1=1) で呼び出してください。

Syntax

- type *_uint8_t*

```
1  lora.ReadFirmwareVersion();
```

Parameters

- none

Returns

- 0x10: ver.1.x;
- 0x20: E220-900T22S R2;
- 0x28: E220-900T22L;
- -1: 失敗

9.2 Methods for ver.2.0

9.2.1 `init_REG09_0A_Byte()`

MCU からの UART 送信データに CHECKSUM 付与が設定されているモジュールであっても、再起動時などでのコンフィグ設定を可能にするため、初期設定の前に CHECKSUM 設定を解除し、レジスタ 09h とレジスタ 0Ah をゼロクリアします。

Syntax

- type *int*

```
1    lora.init_REG09_0A_Byte();
```

Parameters

- none

Returns

- 0:成功; 1: 失敗

9.2.2 `SetConfigStrictMode()`

E220-900T22S(JP) をコンフィグモード (M0=1,M1=1) へ移行させ、`config`の構造体メンバをもとに Strict Mode 設定を行います。

Syntax

- type *int*

```
1    lora.SetConfigStrictMode();
```

Parameters

- none

Returns

- 0:成功; 1: 失敗*** ### `RecieveFrame()` LoRa 受信を行い、引数 `recv_frame`の構造体メンバへ値を格納します。

Syntax

- type *int*

```
1 lora.RecieveFrame(struct RecvFrameE220900T22SJP_t *recv_frame);
```

Parameters

- *recv_frame*: LoRa 受信データの格納先

Returns

- 0:成功; 1: 失敗

9.2.3 Lowvoltage_operation_Enable()

低電圧動作を有効にします。

Syntax

- type *int*

```
1 lora.Lowvoltage_operation_Enable();
```

Parameters

- none

Returns

- 0:成功; 1: 失敗

9.2.4 Lowvoltage_operation_Disable()

低電圧動作を無効にします。

Syntax

- type *int*

```
1 lora.Lowvoltage_operation_Disable();
```

Parameters

- none

Returns

- 0:成功; 1: 失敗

9.2.5 StrictModeStart()

Strict Mode（ver.2.0 厳格動作）を設定します。

Syntax

- type *int*

```
1 lora.StrictModeStart();
```

Parameters

- none

Returns

- 0:成功; 1: 失敗

9.2.6 StrictModeEnd()

Strict Mode（ver.2.0 厳格動作）を解除します。

Syntax

- type *int*

```
1 lora.StrictModeEnd();
```

Parameters

- none

Returns

- 0:成功; 1: 失敗

9.2.7 ReceivePKTaddADRS()

受信パケットに宛先アドレスを出力する設定を行います。

Syntax

- type *int*

```
1 lora.ReceivePKTaddADRS();
```

Parameters

- none

Returns

- 0:成功; 1: 失敗

9.2.8 ReceivePKTdelADRS()

受信パケットに宛先アドレスを出力する設定を解除します。

Syntax

- type *int*

```
1 lora.ReceivePKTdelADRS();
```

Parameters

- none

Returns

- 0:成功; 1: 失敗

9.2.9 SendPKTaddCS()

MCU から UART 送信データ列に CHECKSUM を付与する設定を行います。

Syntax

- type *int*

```
1  lora.SendPKTaddCS();
```

Parameters

- none

Returns

- 0:成功; 1: 失敗

9.2.10 SendPKTdelCS()

MCU から UART 送信データ列に CHECKSUM を付与する設定を解除します。

Syntax

- type *int*

```
1  lora.SendPKTdelCS();
```

Parameters

- none

Returns

- 0:成功; 1: 失敗

9.2.11 ReceivePKTaddCS()

UART での受信列への CHECKSUM を付与する要求を有効にします。

Syntax

- type *int*

```
1 lora.ReceivePKTaddCS();
```

Parameters

- none

Returns

- 0:成功; 1: 失敗

9.2.12 ReceivePKTdelCS()

UART での受信列への CHECKSUM を付与する要求を無効にします。

Syntax

- type *int*

```
1 lora.ReceivePKTdelCS();
```

Parameters

- none

Returns

- 0:成功; 1: 失敗

9.2.13 ReceivePKTaddSIZE()

UART での受信列へのパケットサイズを付与する要求を有効にします。

Syntax

- type *int*

```
1 lora.ReceivePKTaddSIZE();
```


Parameters

- none

Returns

- 0:成功; 1: 失敗

9.2.14 ReceivePKTdelSIZE()

UART での受信列へのパケットサイズを付与する要求を無効にします。

Syntax

- type *int*

```
1 lora.ReceivePKTdelSIZE();
```

Parameters

- none

Returns

- 0:成功; 1: 失敗

9.2.15 REG09_ByteWrite()

lora.REG09_Byte で設定されたデータを 設定レジスタ 0 9 H に書き込みます。

Syntax

- type *int*

```
1 lora.REG09_ByteWrite();
```

Parameters

- none

Returns

- 0:成功; 1: 失敗

9.2.16 REG09_ByteWrite(uint8_t data)

引数 data で設定されたデータを設定レジスタ 0 9 H に書き込みます。

Syntax

- type *int*

```
1 lora.REG09_ByteWrite(uint8_t data);
```

Parameters

- data: 設定レジスタ 0 9 H へ書き込むデータ

Returns

- 0:成功; 1: 失敗

9.2.17 REG0A_ByteWrite()

lora.REG0A_Byte で設定されたデータを 設定レジスタ 0 A H に書き込みます。

Syntax

- type *int*

```
1 lora.REG0A_ByteWrite();
```

Parameters

- none

Returns

- 0:成功; 1: 失敗

9.2.18 REG09_Set_AUX_HoldTime(uint8_t bt)

設定レジスタ 09 H の受信動作の AUX Low レベル待機時間の設定を書き込みます

Syntax

- type *int*

```
1 lora.REG09_Set_AUX_HoldTime(uint8_t bt);
```

Parameters

- *bt*: 設定レジスタ 09 H の bit1,bit0 へ書き込むビットデータ

Returns

- 0:成功; 1: 失敗

9.2.19 REG0A_Set_CarrierSenseTime(uint8_t ms)

ms で渡されたデータを 設定レジスタ 0A H に書き込みます。また、lora.REG0A_Byte にも設定します。

Syntax

- type *int*

```
1 lora.REG0A_Set_CarrierSenseTime(uint8_t ms);
```

Parameters

- *ms*: 設定レジスタ 0A H へ書き込むデータ 0、1～254

Returns

- 0:成功; 1: 失敗

9.2.20 Write_RegisterLIB()

コンフィグレーションモード (mode3) で書き込める設定レジスタ 09 H、0A H レジスタの書き込み

Syntax

- type *int*

```
1 lora.Write_RegisterLIB(const char* str, int str_length, uint8_t  
SetData);
```

Parameters

- *str*: 書き込みコマンドバイト列 (REG09 の場合{0xc0,0x09,0x01},REG0A の場合{0xc0,0x0A,0x01})
- *str_length*: 書き込みコマンドバイト数
- *SetData*: 書き込みデータ

Returns

- 0:成功; 1: 失敗

9.2.21 Read_RegisterLIB()

コンフィグレーションモード (mode3) で読み込めるレジスタを読み込み、値を lora.REGData.data[3] ~あるいは、lora.REGData.REGUniStr.val[0]~に格納します。

Syntax

- type *int*

```
1 lora.Read_RegisterLIB( uint8_t address, uint8_t bytes);
```

Parameters

- *address*: 読み込むREGアドレス
- *bytes*: 読み込むデータ数

Returns

- 0:成功; 1: 失敗

9.2.22 Read_RegisterLIB_s()

通常送信モード (mode0)、WOR 送信モード (mode1) で読み込めるレジスタを読み込み、値を lora.REGData.data[3] ~あるいは、lora.REGData.REGUniStr.val[0]~に格納します。

Syntax

- type *int* `lora.Read_RegisterLIB_s( uint8_t address, uint8_t bytes);`

Parameters

- *address*: 読み込むREGアドレス
- *bytes*: 読み込むデータ数

Returns

- 0:成功; 1: 失敗

9.2.23 read_id_num()

個体番号レジスタを読み込み、lora.REGData.REGUniStr.val[0] に格納します。

Syntax

- type *int*

```
1  lora.read_id_num(char RegNum);
```

Parameters

- *RegNum*: 読み込む個体番号アドレス

Returns

- 0:成功; 1: 失敗

9.2.24 write_id_num()

個体番号レジスタへの書き込み

Syntax

- type *int*

```
1  lora.write_id_num(char RegNum, uint8_t IDdata);
```

Parameters

- *RegNum*: 書き込む個体番号アドレス
- *IDdata*: 書き込むデータ

Returns

- 0:成功; 1: 失敗

9.2.25 Check_BufferedReceiveData_Exist()

レジスタアクセス(読み書き)の前に、UART バッファのデータ有無をチェックし、有りの場合に読み込み LoRa 受信データの格納先に格納します。※ Read_RegisterLIB, Read_Register_s, Write_RegisterLIB には実装済み

Syntax

- type *int*

```
1  lora.Check_BufferedReceiveData_Exist();
```

Parameters

- none

Returns

- 0:データ有り; 1: データ無し